Sentinel: A Functional Signature-Based Virus Scanner

513559004 Jsaon Chia-Sheng Lin; 313264012 Ching-Chung Chen

2026-04-22

Abstract

Sentinel is a safe, read-only, signature-based virus scanner for the Network Security Practice final project. It scans an explicit target directory, loads a structured malware-signature database, combines Bloom-filter hash pre-checks with exact hash-map verification, performs Aho-Corasick byte-pattern matching, skips symbolic links by default, separates heuristic warnings from confirmed signature matches, and generates JSON and Markdown evidence reports. The demo uses only local safe fixtures, not live malware.

1 Team And Submission Decisions

This report covers Project I - Virus Scanner. The recorded team members are 513559004 Jsaon Chia-Sheng Lin and 313264012 Ching-Chung Chen.

The implementation is not moved into a private GitHub or GitLab repository in this pass, by instruction. The private repository URL is therefore recorded as not created or moved. The local demo source-baseline commit hash is `2be51a0f003834e58795efcdd4b9224a730b90e7`. The current final demo does not require the literal EICAR test file; it uses a safe mock fixture plus EICAR reference hashes unless the instructor explicitly requires literal EICAR later.

2 Introduction

The project goal is to identify a known mock threat hidden inside a directory tree and produce a clear security report. Sentinel is intentionally educational: it does not execute scanned files, delete files, quarantine files, upload files, or scan the whole machine by default. The current demo uses a local safe mock-virus marker rather than live malware or the literal EICAR string. If the instructor requires the exact EICAR test file, it should be created only in the controlled final demo environment and recorded in the final evidence manifest.

3 Requirement Mapping

Official requirement	Sentinel implementation
Structured malware-signature repository	`signatures/malware-signatures.json` stores MD5, SHA-256, and hex-pattern matchers.
File traversal and comparison	`sentinel.scanner` walks the explicit target directory in sorted order and scans regular files.
Bitwise or pattern comparison	`hex_pattern` signatures are converted to bytes and searched during chunked file scans using an Aho-Corasick byte-pattern automaton.

Heuristic analysis	`sentinel.heuristics` flags low-confidence suspicious indicators without claiming confirmed infection.
Security report	`sentinel.reporting` writes timestamped JSON and Markdown reports with paths, statuses, severities, hashes, and evidence.
Demonstration	`demo/demo-tree/` contains clean files, one safe mock-virus fixture, and one heuristic-only suspicious fixture.

4 Signature Database Design

The signature database uses JSON because it is easy to inspect, easy to validate with the Python standard library, and flexible enough to support multiple matcher types per signature. Each signature has a stable identifier, name, category, severity, matchers, and optional notes.

The current safe mock-virus signature uses three matchers:

Matcher	Purpose
`md5`	Fast exact-file detection for the known demo fixture.
`sha256`	Stronger exact-file detection for the known demo fixture.
`hex_pattern`	Byte-pattern matching for the bitwise comparison requirement.

4.1 Data Structures

Structure	Purpose	Reason
`tuple[Signature, ...]`	Validated signature records	Keeps the loaded database stable during a scan.
`dict[str, dict[str, tuple[HashMatcher, ...]]]`	Hash lookup by algorithm and digest	Exact hash detection becomes a constant-time dictionary lookup.
`HashBloomFilter`	Hash membership precheck	Avoids unnecessary exact hash-map checks while preserving exact verification for possible hits.
`PatternScanEngine`	Aho-Corasick byte-pattern automaton	Reused across files during a scan run.
`PatternStreamScanner`	Per-file automaton state	Preserves pattern state across streamed chunks.
`list[HeuristicFinding]`	Suspicious-rule output	Keeps heuristic signals separate from confirmed signature matches.

The Bloom filter stores normalized `algorithm:digest` keys for configured hash signatures. It answers only "might contain." Sentinel then confirms every possible hit against the exact hash map, so Bloom-filter false positives cannot create infected results.

5 Scanning Engine

The scanner workflow is:

1. Parse CLI arguments.

2. Load and validate the signature database.
3. Walk the explicit target directory in sorted order.
4. Report symbolic links as `skipped` rather than following them.
5. Read each file in chunks.
6. Compute MD5 and SHA-256 incrementally.
7. Pre-check hash signatures through the Bloom filter.
8. Confirm possible hash hits through exact lookup maps.
9. Search configured byte patterns with an Aho-Corasick automaton.
10. Run heuristic rules against a bounded sample.
11. Assign status: `infected`, `suspicious`, `clean`, `skipped`, or `error`.
12. Write JSON or Markdown report output.

Chunked scanning avoids loading full files into memory for hash and pattern matching. For byte-pattern matching, Sentinel builds a trie with failure links from all configured patterns, then feeds file chunks through one automaton state machine. This means a signature split across two chunks is still detectable without manually searching each pattern against each chunk. The overlapping, stream-boundary, and chunk-boundary pattern tests verify this behavior.

5.1 Algorithmic Hardening

The pattern engine uses the Aho-Corasick idea: multiple patterns are compiled into one automaton, and each input byte advances one shared state machine. The JSON report records `hash_filter`, `hash_filter_policy`, `pattern_engine`, `pattern_count`, `automaton_states`, chunk-size metadata, and `symlink_policy` so the demo evidence shows which matching and traversal paths were used.

The benchmark in `reports/pattern-benchmark.json` is safe and synthetic. It compares the automaton with a naive per-pattern `bytes` search. The current benchmark shows equal match sets. It does not claim production antivirus performance; in small Python-only demos, CPython's C-backed byte search can be faster than a pure-Python automaton. The professional value here is explainable multi-pattern behavior and scaling structure.

The scanner returns exit code `1` when an infected file is found. In the demo this is expected because the safe mock-virus fixture is intentionally detected.

6 Heuristic Analysis

Heuristics are deliberately separated from signatures. A heuristic-only hit is reported as `suspicious`, not `infected`. This avoids overstating weak evidence.

Rule	Signal	Limitation
`api-name`	Safe fixture mentions API names such as `CreateRemoteThread`, `VirtualAllocEx`, and `WriteProcessMemory`.	String presence alone does not prove malware.

`magic-mismatch`	A `.txt` file begins with executable-like `MZ` bytes.	Benign mislabeled files can exist.
`high-entropy`	A small binary-like file has high byte entropy.	Compressed benign files may also trigger.

7 Report And Evidence Artifacts

Sentinel currently generates:

- `reports/demo-report.json`: machine-readable scan report.
- `reports/demo-report.md`: instructor-friendly Markdown report.
- `reports/pattern-benchmark.json`: safe synthetic benchmark data.
- `reports/pattern-benchmark.md`: benchmark summary.
- `reports/demo-evidence-manifest.json`: reproducibility manifest.

The evidence manifest records SHA-256 hashes and sizes for every demo-tree file, the signature database, generated reports, and benchmark artifacts. It also records the safety flags and exact commands used to reproduce the evidence.

8 External Standards Alignment

The project was calibrated against a small set of recognized references: EICAR/CARO for safe anti-malware testing, NIST SP 800-83 Rev. 1 for malware prevention and handling, NIST Cybersecurity Framework 2.0 for risk-management framing, OWASP File Upload guidance for defense-in-depth around untrusted files, and MITRE ATT&CK T1055.002 for process-injection API context. The mapping is recorded in `docs/standards-alignment.md`.

Sentinel uses these references conservatively. The repository stores an EICAR reference signature file containing only the reference MD5 and SHA-256 hashes. It does not store the literal EICAR test file. The EICAR unit tests reconstruct the reference bytes in memory to verify the standard 68-byte length and expected hashes. MITRE API-name indicators are reported only as `suspicious`; they are not treated as confirmed malware without an exact signature match.

9 Release Readiness

Sentinel includes `VERSION`, `CHANGELOG.md`, `Makefile`, and `scripts/check_release.py`. The release check is intended to be run before moving the package into the required private repository and again after the move. It validates version consistency, standards alignment, EICAR reference hashes, full demo regeneration, expected demo counts, Bloom-filter metadata, Aho-Corasick metadata, symbolic-link policy, benchmark equivalence, evidence-manifest safety flags, private-repository export boundaries, and final-report PDF presence.

```
PYTHONDONTWRITEBYTECODE=1 PYTHONPATH=src python3 scripts/check_release.py
```

10 Current Evaluation

10.1 Demo Result

Metric	Result
Files scanned	5
Infected	1
Suspicious	1
Clean	3
Errors	0

The infected file is the nested Sentinel safe mock-virus fixture. It matches by MD5, SHA-256, and hex-pattern evidence. The suspicious file is the API-name fixture; it is marked suspicious because it contains API-name indicators, not because it matches a confirmed malware signature.

The demo report records `bloom-filter` as the hash pre-check and states that possible hits are verified by the exact hash map. It also records `aho-corasick-byte-automaton` as the pattern engine, with one configured pattern and 30 automaton states for the current signature database.

10.2 Test Result

The current standard-library test suite contains 25 tests. It covers signature validation, Bloom-filter hash pre-checks, exact hash matching, Aho-Corasick byte-pattern matching, overlapping pattern outputs, stream/chunk-boundary pattern detection, scanner behavior, symbolic-link skipping, EICAR reference hash validation, report generation, Markdown output, evidence manifest generation, private-repo export boundaries, CLI paths, and the version flag.

```
PYTHONDONTWRITEBYTECODE=1 PYTHONPATH=src python3 -m unittest discover -s tests -v
```

11 Screenshot Evidence

This report includes screenshot evidence for each primary project part: signature database, scanner interface and core metadata, demo folder structure, scan result, release gate, and final submission package.

```
Evidence 1 - Signature database
$ sed -n "1,70p" signatures/malware-signatures.json

{
  "schema_version": "1.0",
  "updated_at": "2026-04-22T00:00:00+08:00",
  "signatures": [
    {
      "id": "sig-sentinel-safe-mock-virus",
      "name": "Sentinel Safe Mock Virus Fixture",
      "category": "safe-mock-virus",
      "severity": "critical",
      "matchers": [
        {
          "type": "md5",
          "value": "8a6029d819465ac7d2778dc324230230"
        },
        {
          "type": "sha256",
          "value": "569f58884e12c423aa9442c7b220c2814ea0febb66278e61f4b8b0bd35dad122"
        },
        {
          "type": "hex_pattern",
          "value": "53454e54494e454c5f534146455f4d4f434b5f56495255535f32303236"
        }
      ],
      "notes": "Course demo fixture only. This is not live malware and should not be represented as a real-world
threat."
    }
  ]
}
```

Figure 1: Signature database evidence showing the structured safe mock-virus signatures.

```
Evidence 2 - Scanner CLI and engine metadata
$ PYTHONPATH=src python3 -m sentinel --help

usage: sentinel [-h] [--version]
                {scan,validate-signatures,summarize-report,write-evidence} ...

Sentinel safe signature-based scanner for the network security project.

positional arguments:
  {scan,validate-signatures,summarize-report,write-evidence}
    scan                Scan an explicit file or directory.
    validate-signatures
                        Validate a JSON signature database.
    summarize-report    Print a compact summary of a report.
    write-evidence      Write a reproducible demo evidence manifest.

options:
  -h, --help            show this help message and exit
  --version             show program's version number and exit

$ rg -n "hash_filter|pattern_engine|symlink_policy|precheck" src/sentinel/{scanner.py,bloom.py,matchers.py}

src/sentinel/matchers.py:51:            "pattern_engine": "aho-corasick-byte-automaton",
src/sentinel/matchers.py:137:        if not database.hash_filter.might_contain(algorithm, digest):
src/sentinel/matchers.py:152:    pattern_engine: PatternScanEngine | None = None,
src/sentinel/matchers.py:167:    engine = pattern_engine or PatternScanEngine(database.patterns)
src/sentinel/bloom.py:49:            "hash_filter": "bloom-filter",
src/sentinel/bloom.py:50:            "hash_filter_items": self.item_count,
src/sentinel/bloom.py:51:            "hash_filter_bits": self.bit_count,
src/sentinel/bloom.py:52:            "hash_filter_hash_functions": self.hash_function_count,
src/sentinel/bloom.py:53:            "hash_filter_policy": "precheck-then-exact-hash-map",
src/sentinel/scanner.py:19:    pattern_engine = PatternScanEngine(database.patterns)
src/sentinel/scanner.py:21:        **database.hash_filter.metadata(),
src/sentinel/scanner.py:22:        **pattern_engine.metadata(),
src/sentinel/scanner.py:24:        "symlink_policy": "skip",
src/sentinel/scanner.py:64:        results.append(scan_file(root, root, database, pattern_engine))
src/sentinel/scanner.py:77:                results.append(scan_file(path, root, database, pattern_engine))
src/sentinel/scanner.py:94:    pattern_engine: PatternScanEngine | None = None,
src/sentinel/scanner.py:99:        content = scan_file_content(path, database, pattern_engine=pattern_engine)
```

Figure 2: Scanner interface and engine metadata evidence showing the CLI and implemented hash/pattern policies.

```
Evidence 3 - Complex demo folder structure
$ find demo/demo-tree -maxdepth 4 -type f | sort

demo/demo-tree/clean/image-placeholder.bin
demo/demo-tree/clean/notes.txt
demo/demo-tree/ignored-or-empty/empty.txt
demo/demo-tree/nested/level-1/level-2/sentinel-safe-mock-virus.txt
demo/demo-tree/suspicious/api-names-fixture.txt
```

Figure 3: Demo folder evidence showing clean files, nested safe mock-virus fixture, and suspicious fixture.

```
Evidence 4 - Demo scan result report
$ sed -n "1,45p" reports/demo-report.md

# Sentinel Scan Report

## Run Metadata

- Tool: `Sentinel`
- Target: `demo/demo-tree`
- Signature schema: `1.0`
- Started: `2026-04-22T06:47:50+00:00`
- Finished: `2026-04-22T06:47:50+00:00`

## Scan Engine

- Hash pre-check: `bloom-filter`
- Hash filter items: `2`
- Hash filter bits: `128`
- Hash filter policy: `precheck-then-exact-hash-map`
- Pattern engine: `aho-corasick-byte-automaton`
- Pattern count: `1`
- Automaton states: `30`
- Chunk size bytes: `1048576`
- Symlink policy: `skip`

## Summary

| Metric | Count |
| --- | ---: |
| Files scanned | 5 |
| Infected | 1 |
| Suspicious | 1 |
| Clean | 3 |
| Skipped | 0 |
| Errors | 0 |

## Findings

| Path | Status | Severity | Evidence |
| --- | --- | --- | --- |
| `nested/level-1/level-2/sentinel-safe-mock-virus.txt` | infected | critical | md5:sig-sentinel-safe-mock-virus;
sha256:sig-sentinel-safe-mock-virus; hex_pattern:sig-sentinel-safe-mock-virus@109 |
| `suspicious/api-names-fixture.txt` | suspicious | medium | api-name-indicator (CreateRemoteThread, VirtualAllocEx,
 WriteProcessMemory) |

## All Results

| Path | Status | Severity | Size | SHA-256 |
| --- | --- | --- | ---: | --- |
... truncated for screenshot; source file contains the full output.
```

Figure 4: Demo report evidence showing the scan metadata, summary, and finding rows.

```
Evidence 5 - Release verification gate
$ python3 scripts/check_release.py

[ ok ] version files agree
[ ok ] standards alignment is present
[ ok ] EICAR reference hashes are consistent
[ ok ] demo regeneration passes
[ ok ] demo report is consistent
[ ok ] benchmark evidence is consistent
[ ok ] evidence manifest is consistent
[ ok ] private repo export plan is consistent
[ ok ] final report PDFs exist

Release check passed for Sentinel 0.4.0.
```

Figure 5: Release gate evidence showing the local release-readiness check passing.

```
Evidence 6 - Submission package and final deliverables
$ sed -n "1,42p" report/submission-package.md

# Project I Submission Package

## Purpose

This file is the single last-mile source for final submission. It combines the
local package checklist, private-repository handoff, verification commands, and
safety checks so submission state does not split across multiple notes.

## Current Local State

| Area | State | Evidence |
| --- | --- | --- |
| Scanner package | Working | `src/sentinel/`, `pyproject.toml` |
| Signature database | Working | `signatures/malware-signatures.json` |
| Demo tree | Working | `demo/demo-tree/` |
| Tests | Passing, `25` tests | `python3 demo/run_demo.py` |
| Reports | Generated | `reports/demo-report.json`, `reports/demo-report.md` |
| Benchmark | Generated | `reports/pattern-benchmark.json`, `reports/pattern-benchmark.md` |
| Evidence manifest | Generated | `reports/demo-evidence-manifest.json` |
| Private repo export | Working | `scripts/export_private_repo.py` |
| Final report | Compiled, `6` pages; screenshot-evidence copy compiled as `12` pages | `report/` |
| Release gate | Passing locally | `python3 scripts/check_release.py` |

Latest verified result on `2026-04-22`: Sentinel `v0.4.0`, `5` files scanned,
`1` infected safe mock fixture, `1` suspicious fixture, `3` clean files,
`0` skipped files, and `0` errors.

## Final Project Decisions

| Field | Recorded value |
| --- | --- |
| Project package | Project I - Virus Scanner final submission package. |
| Team members | `513559004` Jsaon Chia-Sheng Lin; `313264012` 陳靖中 (Ching-Chung Chen) |
| Private repository URL | Not created or moved in this pass, per instruction. |
| Local demo source-baseline commit hash | `2be51a0f003834e58795efcdd4b9224a730b90e7` |
| Literal EICAR decision | Not required for the current final demo. Use the safe mock fixture plus EICAR reference
hashes; create literal EICAR only if the instructor explicitly requires it later. |
| Demo format | Live-demo script/runbook prepared; short video not recorded in this pass. |
```

Figure 6: Submission package evidence showing the final deliverable checklist and recorded team fields.

12 Reproduction Commands

The main reproducibility path is:

```
python3 demo/run_demo.py
```

This command runs the tests, validates both signature files, regenerates JSON and Markdown scan reports, regenerates the synthetic benchmark, and regenerates the evidence manifest. The final consistency check is:

```
python3 scripts/check_release.py
```

Before moving the package into the required private repository, the curated handoff folder can be generated with:

```
python3 scripts/export_private_repo.py --clean
```

The project includes `pyproject.toml`, so it can also be installed in editable mode with `python3 -m pip install -e .`. After installation, the `sentinel` console command is available for the same scan, validation, summary, and evidence-manifest workflows.

13 Limitations And Ethics

Sentinel is not production antivirus software. It detects only signatures and simple heuristic indicators included in the configured database and rules. It does not execute files, detonate samples, collect telemetry, or process live malware. The current mock-virus fixture demonstrates scanner mechanics rather than real-world malware coverage. The repository does not store the literal EICAR test file; it stores only reference hashes and in-memory tests. The benchmark is safe synthetic evidence for algorithm explanation, not a production antivirus performance claim.